PADCOM V22 - GUIA FINAL DE IMPLEMENTAçãO

Versão: 1.0

Data: 11 de Abril de 2026

Status: Pronto para Implementação

VISÃO GERAL DO SISTEMA

O PADCOM V22 é um sistema completo de **check-in e acompanhamento longitudinal** para medicina integrativa, com:

- **Entrada única**: Paciente marca doenças, cirurgias, cânceres e sintomas (não responde do zero)
- **Acompanhamento automático**: Cards mensais para Admin, Enfermeira, Consultor
- **Timeline completa**: Histórico de tudo que foi perguntado e respondido
- **Dashboard médico**: Visão consolidada com correlação de exames e evolução
- **Motor clínico**: Processamento automático de sugestões

ARQUIVOS ENTREGUES

Arquivo	Descrição	Tipo
`drizzle_schemas_v22.ts`	Schemas Drizzle ORM	TypeScript
`migrations_v22.sql`	Migrations PostgreSQL	SQL
`routes_v22_checkin.ts`	Rotas Express	TypeScript
`components_v22_ui.tsx`	Componentes React	TypeScript/React
`validations_v22.ts`	Validações Zod	TypeScript
`seed_v22_complete.mjs`	Script de seed	JavaScript
`routes_v22_checkin.test.ts`	Testes unitários	TypeScript
`openapi_v22.yaml`	Documentação API	YAML

`SECURITY_PERFORMANCE_V22.md`	Segurança e Performance	Markdown

FASES DE IMPLEMENTAÇÃO

FASE 1: PREPARAÇÃO DO BANCO DE DADOS

Objetivo: Criar tabelas e estrutura do banco

Passos:

1. Executar migrations:

   Bash
   ```bash
   npm run migrate -- migrations_v22.sql
   ```

2. Verificar tabelas criadas:

   SQL
   ```sql
   SELECT table_name FROM information_schema.tables
   WHERE table_schema = 'public';
   ```

3. Executar seed com dados iniciais:

   Bash
   ```bash
   node seed_v22_complete.mjs
   ```

Resultado esperado: 14 tabelas criadas com dados de patologias, cirurgias, cânceres e sintomas.

FASE 2: INTEGRAÇÃO DE SCHEMAS

Objetivo: Adicionar schemas Drizzle ao projeto

Passos:

1. Copiar `drizzle_schemas_v22.ts` para `lib/db/src/schema/v22/` :

   Bash

```
cp drizzle_schemas_v22.ts lib/db/src/schema/v22/index.ts
```

2. Atualizar `lib/db/src/index.ts` para exportar novos schemas:

```TypeScript
export * from "./schema/v22/index";
```

3. Executar type check:

```Bash
npm run check
```

Resultado esperado: Schemas compilam sem erros.

FASE 3: IMPLEMENTAÇÃO DE ROTAS

Objetivo: Adicionar rotas Express para check-in e acompanhamento

Passos:

1. Copiar `routes_v22_checkin.ts` para `artifacts/api-server/src/routes/` :

```Bash
cp routes_v22_checkin.ts artifacts/api-server/src/routes/v22.ts
```

2. Registrar rotas em `artifacts/api-server/src/index.ts` :

```TypeScript
import v22Routes from "./routes/v22";
app.use("/api", v22Routes);
```

3. Copiar `validations_v22.ts` para `artifacts/api-server/src/lib/` :

```Bash
cp validations_v22.ts artifacts/api-server/src/lib/validations.ts
```

4. Testar rotas:

```Bash
```

```
npm run dev
# Acessar http://localhost:3000/api/v22/patologias-master
```

Resultado esperado: Rotas respondendo com dados corretos.

FASE 4: COMPONENTES REACT

Objetivo: Criar UI para check-in e dashboard

Passos:

1. Copiar `components_v22_ui.tsx` para `client/src/components/v22/` :

   Bash
   ```bash
   mkdir -p client/src/components/v22
   cp components_v22_ui.tsx client/src/components/v22/index.tsx
   ```

2. Criar página de check-in em `client/src/pages/CheckinV22.tsx` :

   TypeScript
   ```typescript
   import { CheckinPatologiasComponent } from "@/components/v22";

   export default function CheckinV22Page( ) {
     return <CheckinPatologiasComponent />;
   }
   ```

3. Criar página de dashboard em `client/src/pages/DashboardV22.tsx` :

   TypeScript
   ```typescript
   import { DashboardComponent } from "@/components/v22";

   export default function DashboardV22Page() {
     return <DashboardComponent />;
   }
   ```

4. Adicionar rotas em `client/src/App.tsx` :

   TypeScript
   ```typescript
   <Route path="/v22/checkin" component={CheckinV22Page} />
   <Route path="/v22/dashboard/:pacienteId" component={DashboardV22Page} />
   ```

Resultado esperado: UI funcional para check-in e dashboard.

FASE 5: TESTES

Objetivo: Validar funcionalidade do sistema

Passos:

1. Copiar `routes_v22_checkin.test.ts` para `artifacts/api-server/src/routes/` :

   Bash
   ```bash
   cp routes_v22_checkin.test.ts artifacts/api-server/src/routes/v22.test.ts
   ```

2. Executar testes:

   Bash
   ```bash
   npm run test -- v22.test.ts
   ```

3. Verificar cobertura:

   Bash
   ```bash
   npm run test -- --coverage
   ```

Resultado esperado: Testes passando com cobertura > 80%.

FASE 6: MOTOR CLÍNICO

Objetivo: Integrar processamento automático

Passos:

1. Implementar função `processarCheckinMotor()` em `routes_v22_checkin.ts` :

   TypeScript
   ```typescript
   async function processarCheckinMotor(
     pacienteId: number,
     anamneseId: number
   ) {
     // 1. Buscar patologias atuais
     const patologias = await db
       .select()
       .from(pacientePatologiasCheckinTable)
   ```

```
    .where(eq(pacientePatologiasCheckinTable.pacienteId, pacienteId));

  // 2. Aplicar regras do motor
  const sugestoes = await aplicarRegrasMotor(patologias);

  // 3. Retornar sugestões
  return sugestoes;
}
```

2. Implementar geração de cards mensais:

```typescript
async function gerarCardsMensais(pacienteId: number, mesAno: Date) {
  // 1. Gerar 3 cards (Admin, Enfermeira, Consultor)
  // 2. Selecionar perguntas baseadas no histórico
  // 3. Salvar no banco
}
```

Resultado esperado: Motor processando check-ins e gerando cards.

FASE 7: SEGURANÇA E PERFORMANCE

Objetivo: Implementar segurança e otimizações

Passos:

1. Ler `SECURITY_PERFORMANCE_V22.md` completamente
2. Implementar validação Zod em todas as rotas:

```typescript
router.post(
  "/v22/checkin/patologias",
  validateBody(CheckinPatologiasRequestSchema),
  checkinPatologiasHandler
);
```

3. Implementar autenticação JWT:

```typescript
router.get(
  "/v22/dashboard/paciente/:pacienteId",
  auth.middleware,
```

```
  dashboardHandler
);
```

4. Configurar rate limiting:

```typescript
app.use("/api/v22/", rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));
```

5. Adicionar índices do banco:

```sql
CREATE INDEX idx_paciente_patologias_checkin_paciente_id
  ON paciente_patologias_checkin(paciente_id);
```

Resultado esperado: Sistema seguro e otimizado.

FASE 8: DOCUMENTAÇÃO E DEPLOY

Objetivo: Documentar e preparar para produção

Passos:

1. Gerar documentação Swagger:

```bash
npm run swagger -- openapi_v22.yaml
```

2. Acessar em: `http://localhost:3000/api-docs`
3. Executar checklist de segurança (em `SECURITY_PERFORMANCE_V22.md`)
4. Executar checklist de performance
5. Deploy em staging:

```bash
npm run build
npm run start
```

6. Testes de carga:

```bash
```

```
npm run load-test
```

Resultado esperado: Sistema pronto para produção.

ESTRUTURA DE DADOS

Tabelas Principais

Tabela	Descrição	Campos Chave
`patologias_master`	Catálogo de doenças	id, codigoPadcom, nome, grupo
`paciente_patologias_checkin`	Doenças do paciente	pacienteId, patologiaId, intensidade0_5
`acompanhamento_cards_mensais`	Cards de follow-up	pacienteId, mesAno, tipoCard, status
`acompanhamento_timeline`	Histórico de eventos	pacienteId, dataEvento, tipoEvento
`perguntas_cards_template`	Perguntas dos cards	tipoCard, pergunta, tipo

Relacionamentos

Plain Text

```
pacientes (1) ──→ (N) paciente_patologias_checkin ←─ (N) patologias_master
pacientes (1) ──→ (N) acompanhamento_cards_mensais
pacientes (1) ──→ (N) acompanhamento_timeline
```

FLUXO DE DADOS

1. Check-in (Entrada)

Plain Text

```
Paciente marca doenças
    ↓
POST /api/v22/checkin/patologias
    ↓
Validar com Zod
    ↓
Salvar em paciente_patologias_checkin
    ↓
Criar evento na timeline
    ↓
Processar no motor clínico
    ↓
Retornar sugestões
```

2. Acompanhamento (Engine)

Plain Text

```
1º dia do mês
    ↓
Gerar 3 cards (Admin, Enfermeira, Consultor)
    ↓
Selecionar perguntas baseadas no histórico
    ↓
Salvar em acompanhamento_cards_mensais
    ↓
Notificar responsáveis
```

3. Dashboard (Output)

Plain Text

```
Médico acessa /v22/dashboard/paciente/:id
    ↓
Buscar patologias atuais
    ↓
Buscar últimos sintomas
    ↓
Buscar cards pendentes
    ↓
Buscar timeline recente
    ↓
Retornar dashboard consolidado
```

ENDPOINTS PRINCIPAIS

Master Data

Plain Text

```
GET /api/v22/patologias-master
GET /api/v22/cirurgias-master
GET /api/v22/canceres-master
GET /api/v22/sintomas-master
```

Check-in

Plain Text

```
POST /api/v22/checkin/patologias
POST /api/v22/checkin/cirurgias
POST /api/v22/checkin/canceres
POST /api/v22/tracking/sintomas
```

Acompanhamento

Plain Text

```
GET /api/v22/acompanhamento/cards/:pacienteId
POST /api/v22/acompanhamento/cards/:cardId/responder
GET /api/v22/acompanhamento/timeline/:pacienteId
```

Dashboard

Plain Text

```
GET /api/v22/dashboard/paciente/:pacienteId
```

Motor Clínico

Plain Text

```
POST /api/v22/motor-clinico/processar-checkin
POST /api/v22/motor-clinico/gerar-cards-mensais
```

VARIÁVEIS DE AMBIENTE

```bash
# .env
DATABASE_URL=postgresql://user:password@localhost:5432/padcom
JWT_SECRET=seu_secret_muito_seguro_aqui
API_KEY=sua_api_key_aqui
NODE_ENV=production
REDIS_URL=redis://localhost:6379
LOG_LEVEL=info
```

TROUBLESHOOTING

Erro: "Tabelas não existem"

Solução: Executar migrations

```bash
npm run migrate -- migrations_v22.sql
```

Erro: "Validação falhou"

Solução: Verificar payload contra schema em `validations_v22.ts`

Erro: "Paciente não encontrado"

Solução: Verificar se pacienteId existe no banco

Performance lenta

Solução: Verificar índices e executar ANALYZE

```sql
ANALYZE;
```

PRÓXIMOS PASSOS

1. ✅ Implementar as 8 fases
2. ✅ Executar testes
3. ✅ Validar segurança
4. ✅ Deploy em staging
5. ✅ Testes com pacientes piloto
6. ✅ Feedback e ajustes
7. ✅ Deploy em produção

SUPORTE

Para dúvidas ou problemas:

1. Consultar `SECURITY_PERFORMANCE_V22.md`
2. Consultar `openapi_v22.yaml` para detalhes de API
3. Executar testes: `npm run test`
4. Verificar logs: `tail -f logs/app.log`

Sistema pronto para implementação. Boa sorte! 🚀